

Auto-Indexing for PostgreSQL

Project Summary Report

Team: Kabir S Prakash (23B1036), Jainendrajeet (23B1008)
Somya Dahiaya (23B1025), Udit Sarkar (23B1009)

Date: Thursday 30th April, 2026

1 Motivation

The core motivation behind this project is that index design in real database systems is not a trivial administrative task. Effective index selection requires a strong understanding of query workloads, table sizes, access patterns and the trade-off between faster reads and the overhead introduced by index maintenance. In practice, a database administrator or developer must repeatedly inspect slow queries, interpret execution plans, identify useful candidate columns, create indexes manually and then validate whether those indexes truly improve performance. This process is time-consuming, highly skill-dependent and difficult to sustain as workloads evolve over time.

We therefore wanted to build a system that reduces this manual burden by allowing PostgreSQL to react to observed workload behavior in a more intelligent way. Instead of relying entirely on static human decisions, our approach continuously observes repeated query patterns, estimates whether a hypothetical index would reduce planner cost and accumulates evidence that an index would be beneficial. Only when that estimated benefit becomes large enough relative to the expected overhead of index creation do we allow the system to create the real index. In this way, the project is motivated not just by automation for its own sake, but by the practical goal of making database tuning more adaptive, less expert-dependent and more responsive to real query behavior.

2 High-Level Design

The system architecture is divided into two decoupled layers:

1. **PostgreSQL Extension (`pg_auto_index`):** A lightweight C-module that captures `SELECT` queries via execution hooks and logs them to a CSV interface.
2. **Python Daemon (`pg_auto_indexer.py`):** An external analyzer that processes the workload, utilizes **HypoPG** for cost estimation and executes DDL commands based on a cost-driven policy.

By leveraging PostgreSQL's built-in query ID and query jumbling mechanism, we were able to group structurally similar queries together without implementing our own fingerprinting logic. This allowed us to identify repeated query patterns in a way that remains compatible with newer PostgreSQL versions while keeping our implementation simpler and more closely aligned with existing PostgreSQL internals.

3 Functionality Implemented

The prototype supports the following end-to-end lifecycle:

- **Query Capture:** High-performance logging of query IDs, timestamps, and planner costs.
- **Heuristic Analysis:** Candidate index extraction from filter, join, and sort conditions.
- **Cost Simulation:** Evaluation of hypothetical indexes without physical storage overhead.
- **Lifecycle Management:** Score-based index creation (ski-rental algorithm) and automatic removal of stale indexes to prevent "index bloat."

4 Project Components

4.1 New Modules Created

Component	Description
pg_auto_index.c	The C source for the extension using ExecutorEnd_hook.
pg_auto_indexer.py	The primary logic engine (daemon) for index management.

4.2 Modified Infrastructure

Minimal changes were made to the existing PostgreSQL build system to ensure our module is compiled seamlessly:

- postgres/contrib/Makefile and meson.build (Registration of the module).
- auto_index_tests/: Developed a suite of automated Python-based test cases.

5 How the System Works

5.1 PostgreSQL-Side Logging

The system was implemented on **PostgreSQL 19** and consists of two main parts: the PostgreSQL-side logger in `pg_auto_index.c` and the Python decision daemon `pg_auto_indexer.py`. We integrated the logger as a **contrib extension** by adding the `pg_auto_index` directory to the relevant `Meson.build` and `Makefile` files.

The logging mechanism is based on PostgreSQL's existing **hook infrastructure**. PostgreSQL already provides the rule that if `ExecutorEnd_hook` is set, the hook function is called whenever execution of a statement finishes. We register our hook during `_PG_init()`, so it is automatically invoked with a `QueryDesc` object as argument.

From `QueryDesc`, we extract the fields needed for workload logging. We first check `queryDesc->operation == CMD_SELECT` so that only **SELECT statements** are considered. We then use `queryDesc->sourceText` for the original SQL text, `queryDesc->plannedstmt->queryId` for the query identifier, and `queryDesc->planstate->plan->total_cost` for the planner's estimated total cost.

We also call `EnableQueryId()` in `_PG_init()` to ensure that PostgreSQL generates **query IDs**. Since PostgreSQL already uses **query jumbling**, structurally similar query shapes receive the

same query ID. This allows us to group repeated queries without implementing a separate fingerprinting mechanism.

5.2 Daemon State and Log Processing

The second part of the system is the Python daemon `pg_auto_indexer.py`. Execution begins at `main`, which first calls `parse_args()` to read command-line arguments. The `AutoIndexer` initializer then connects to the database and creates the main in-memory structures: `states`, which stores `QueryState` objects for each query shape, `indexes`, which stores `IndexState` objects for created indexes, and `table_rows`, which caches row-count estimates for tables.

Before processing begins, `prepare_log_file()` clears the workload log for the current session. The execution flow then proceeds as `main -> run -> tail`. The `tail()` function continuously reads the CSV log file in an infinite loop. When no new rows are available, it calls `drop_stale_indexes()` to evaluate old indexes, waits briefly, and then continues monitoring the file. For each valid CSV row, it emits the parsed row using `yield`, and `run()` passes that row to `process()`.

5.3 Index Selection and Creation Logic

The central decision-making logic is implemented in `process()`. For each logged query, it first checks whether a corresponding `QueryState` already exists. If the query is new, the daemon calls `inspect_query()`, which runs `EXPLAIN (FORMAT JSON)` and uses HypoPG to evaluate possible hypothetical indexes. If the query has been seen before, the daemon updates its existing state by first applying **score decay** and then increasing the score according to the previously computed benefit of the best candidate index.

The function `best_hypothetical_index()` identifies the most useful candidate index. It builds candidate indexes using `index_candidates()`, which generates combinations and permutations of relevant columns extracted from the query plan. For each candidate, HypoPG creates a **hypothetical index**, the query is explained again, and the planner cost is compared with the original cost. The function returns the `best_table`, `best_columns`, and `best_saving`, where `best_saving` is the maximum planner-cost reduction among all tested candidates.

Once the best candidate is known, `process()` compares the accumulated query score with a threshold returned by `index_creation_cost()`. The function first estimates a base threshold using `index_cost_factor * table_rows[table]`, where the row estimate is obtained from `pg_class` if it is not already cached. It then increases this threshold using two **additional penalty** terms: one based on the number of existing indexes already present on the table, and another based on how write-heavy the table is, using statistics from `pg_stat_user_tables`. As a result, tables that already have several indexes or experience more insert, update, and delete activity require stronger evidence before a new index is created. If the accumulated query score exceeds this adjusted threshold, the daemon creates the real index. In this way, the system attempts to create indexes only when the expected benefit is large enough to justify both the indexing overhead and the additional maintenance cost on heavily indexed or write-intensive tables.

5.4 Overall Working Principle

Overall, the system first logs repeated **SELECT query shapes** inside PostgreSQL, then groups similar queries using PostgreSQL's built-in **query jumbling** mechanism, and finally applies a **score-based policy** driven by HypoPG-estimated savings to decide when indexes should be

created or dropped. This keeps the implementation practical, relatively simple, and compatible with modern PostgreSQL internals.

6 Testing and Validation

Validation was performed using a deterministic dataset (`demo_orders`). Our automated testing script, `run_auto_index_demo.py`, validates the workflow by reporting:

- **Optimization Ratio:** Query plan cost reduction before and after indexing.
- **Execution Speedup:** Actual wall-clock time improvements.
- **Selectivity:** Ensuring indexes are created only for high-impact workloads.

On an average auto with auto-indexing we are able to achieve speedup between 30 to 70 times the previous execution cost for the same query ID.

7 Use of AI in the Project

AI was utilized strictly as a **Technical Debugging Assistant**. It aided in navigating the deep call-stack of the PostgreSQL hook system and resolving CSV-parsing edge cases (such as multi-line SQL normalization).

We have used AI to build this project final report for better formatting and structure. AI also assisted in creating `meson.build` and `Makefile` to integrate `pg_auto_index` in the build of custom postgres.

Example Prompt: “We are seeing internal catalog queries in our workload log even though we only want application SELECT statements. What is the likely reason?”
The resulting logic—filtering queries targeting the `pg_catalog` schema—was manually implemented and verified.

Example Prompt: “Above given is my understanding of complete project. All this I want to add in my report under section how to system works. So give me text about this in better format and wordings. You can also elaborate a bit to improve understanding. Give me text here only”

Example Prompt: “Why is an index not being created even after running the same query multiple times manually”

Example Prompt: “Why are multi-line SQL queries in the log file not leading to index creation”

Example Prompt: “Why is the daemon creating an index and then dropping it immediately afterward”

8 Conclusion

This project demonstrates that sophisticated database features like auto-indexing can be implemented modularly via extensions. By porting legacy concepts to a modern PostgreSQL environment using a hook-based architecture, we achieved a balance between powerful automation and system maintainability. Our prototype serves as a functional foundation for future work in temporal indexing and more complex multi-column strategies.